🧾 FICHE MÉMO : Les boucles en Python

🎯 Objectif

Les boucles permettent de **répéter des instructions automatiquement**, sur des séquences (`range`, `string`, `list`) ou tant qu'une condition est vraie.

1️⃣ Boucles avec `range()`

- `range()` génère une **suite de nombres** à parcourir avec `for`.
- Trois formes principales :

Forme	Exemple	Résultat
`range(stop)`	`range(5)`	0, 1, 2, 3, 4
`range(start, stop)`	`range(2,6)`	2, 3, 4, 5
`range(start, stop, step)`	`range(1,10,2)`	1, 3, 5, 7, 9

Exemples pratiques :

```
# itérer de 0 à 4
for i in range(5):
    print(i)

# itérer de 2 à 5
for i in range(2, 6):
    print(i)

# itérer de 1 à 9 par pas de 2
for i in range(1, 10, 2):
    print(i)

# compter à rebours
for i in range(10, 0, -1):
    print(i)
```

```
# while équivalent avec range
i = 0
while i < 5:
    print(i)
    i += 1

# créer une liste avec range
liste = list(range(0, 10, 3))
print(liste)  # [0,3,6,9]
```

2️⃣ Boucles avec des `string`

- On peut **parcourir chaque caractère** d'une chaîne avec `for`.
- Avec `while`, on utilise un index.

Exemples :

```
# for sur string
mot = "Python"
for lettre in mot:
    print(lettre)

# while sur string
mot = "Python"
i = 0
while i < len(mot):
    print(mot[i])
    i += 1
```

3️⃣ Boucles avec des `list`

- On peut parcourir chaque élément d'une liste avec `for`.
- Avec `while`, on utilise un index.

Exemples :

```
# for sur list
fruits = ["pomme", "banane", "kiwi"]
for fruit in fruits:
    print(fruit)

# while sur list
fruits = ["pomme", "banane", "kiwi"]
i = 0
while i < len(fruits):
    print(fruits[i])
    i += 1
```

4️⃣ Mots-clés spéciaux

Mot-clé	Utilité
`break`	Sortir **immédiatement** de la boucle
`continue`	Passer à l'itération suivante
`else`	S'exécute si la boucle **se termine normalement** (pas via `break`)

Exemple :

```
for i in range(5):
    if i == 3:
        break
    if i == 1:
        continue
    print(i)
# affiche : 0, 2
```

5️⃣ Bonnes pratiques

- Choisir `for` pour **parcourir une séquence**
- Choisir `while` pour **condition dépendante**
- Éviter les boucles infinies
- Utiliser `break` et `continue` avec parcimonie

- Pour de grandes séquences, privilégier `range()` plutôt qu'une liste complète

---

6️⃣ Indentation et incrémentation / décrémentation

🔷 Indentation

- En Python, **l'indentation définit le bloc de code** associé à une boucle ou une condition.
- Toutes les **instructions à répéter** doivent être **alignées au même niveau**.
- Une mauvaise indentation provoque une **erreur de syntaxe**.

Exemple correct :

```
i = 0
while i < 3:
    print(i)
    i += 1
```

Exemple incorrect (erreur) :

```
i = 0
while i < 3:
print(i)  # ❌ Indentation manquante
i += 1
```

🔷 Incrémentation

- Pour éviter les **boucles infinies**, il faut **modifier la variable de contrôle**.
- On utilise généralement `i += 1` ou `i = i + 1`.
- Il est important de **ne pas doubler l'incrémentation** sans raison, sinon certaines valeurs sont sautées.

Exemple :

```
i = 0
while i < 5:
```

```
    print(i)
    i += 1  # incrémentation de 1
```

- Résultat : `0, 1, 2, 3, 4`

🔷 Décrémentation

- Pour **compter à l'envers**, on **diminue** la variable de contrôle avec `i -= 1` ou `i = i - 1`.

Exemple :

```
i = 5
while i > 0:
    print(i)
    i -= 1  # décrémentation de 1
```

- Résultat : `5, 4, 3, 2, 1`
- On peut aussi utiliser `range(start, stop, step)` avec `step` négatif pour les boucles `for` :

```
for i in range(5, 0, -1):
    print(i)
# affiche : 5, 4, 3, 2, 1
```
